CRM DE ATENDIMENTO E ORÇAMENTOS

# Roteiro da Apresentação

## Arquitetura e estrutura do backend

O que mostrar, em que ordem e o que falar em cada parte

1 2 3 4 5 6 7

7 seções, na ordem pedida pelo professor

MAPA DA APRESENTAÇÃO

# A ordem do que apresentar

| | Seção | O que deixar na tela | Tempo |
|---|---|---|---|
| 1 | **Introdução e visão geral** | VS Code: árvore de pastas e pyproject.toml | **2 min** |
| 2 | **Domínio rico e Clean Architecture** | VS Code: domain/, orcamento.py, test_arquitetura.py | **4 min** |
| 3 | **Banco de dados e Alembic** | pgAdmin: tabelas · VS Code: alembic/versions | **3 min** |
| 4 | **Rotas e Swagger** | Swagger: rotas, exemplos e um erro 422 | **3 min** |
| 5 | **Autenticação e segurança** | Swagger: login, Authorize, 401 e 403 | **3 min** |
| 6 | **Qualidade e cobertura de testes** | Terminal: testes · navegador: htmlcov | **3 min** |
| 7 | **Destaques da arquitetura** | docker-compose.yml, Dockerfile e docker compose ps | **3 min** |

**Total: cerca de 20 minutos**

Os tempos são sugestão. Se o professor tiver pouco tempo, priorize as seções 2 (o coração do projeto), 5 (segurança) e 6 (testes), e faça a demonstração ao vivo do slide 19.

PREPARAÇÃO

# Antes de o professor chegar

```
# 1. Abra o Docker Desktop e espere "Engine running"
# 2. Na raiz Projeto_CRM, no PowerShell:
docker compose up -d
docker compose ps        # db, api e pgadmin: running
```

### Confira antes

- O login do admin funciona no Swagger.
- Existe pelo menos 1 cliente cadastrado, para o pgAdmin não aparecer vazio.
- Rode os testes uma vez: a 1ª execução demora mais (monta a imagem).

**Deixe abertas no navegador / na tela:**

| | | |
|---|---|---|
| 1 | **VS Code** | pasta Projeto_CRM aberta |
| 2 | **Swagger** | localhost:8000/docs, já logado |
| 3 | **pgAdmin** | localhost:5050, tabela cliente |
| 4 | **Cobertura** | backend/htmlcov/index.html |

**Comandos prontos no slide 21**

O QUE FALAR

*Não há fala aqui — é a checagem. Se algo não subir, "docker compose logs api" mostra o erro. Ter tudo aberto antes evita esperar a tela carregar na frente do professor.*

# A estrutura do repositório

```
Projeto_CRM/
├── docker-compose.yml      # orquestra banco + API + pgAdmin
├── docker/                 # init.sql e pgAdmin
├── docs/                   # requisitos, diagramas, plano
└── backend/
    ├── Dockerfile
    ├── pyproject.toml      # dependências, lint, tipos, cobertura
    ├── pytest.ini
    ├── alembic/            # migrations do banco
    ├── src/                # código da aplicação
    │   ├── domain/         # regras de negócio
    │   ├── application/    # casos de uso
    │   ├── adapters/       # rotas, schemas, repositórios
    │   ├── infra/          # banco, segurança, configuração
    │   └── main.py         # monta a aplicação
    └── tests/              # suíte de testes
        ├── unit/  integration/  e2e/
```

**MOSTRE: árvore de pastas no VS Code**

**2.561** linhas de código em src/

**3.642** linhas de testes em tests/

**4** camadas em src/

**O QUE FALAR**

*"O backend fica em backend/, separado em duas partes: src/, com o código da aplicação, e tests/, com os testes. Dentro de src/ a divisão segue a Clean Architecture, em quatro camadas. Temos mais linhas de teste do que de código — isso já mostra o peso que demos à qualidade."*

# Um arquivo para dependências, testes e lint

```
[project]
dependencies = ["fastapi", "sqlalchemy", "alembic",
                "pydantic-settings", "pg8000", "pyjwt", ...]

[dependency-groups]
dev = ["pytest", "pytest-cov", "mypy", "ruff", ...]

[tool.mypy]
strict = true                # tipagem estrita

[tool.ruff]
line-length = 100            # lint + formatação

[tool.coverage.report]
fail_under = 80              # meta de cobertura
```

**MOSTRE: backend/pyproject.toml**

| | |
|---|---|
| **pytest + pytest-cov** | testes e cobertura |
| **ruff** | lint e formatação |
| **mypy --strict** | checagem de tipos |

**Sem requirements.txt: uma fonte só**

**O QUE FALAR**

*"Tudo é configurado no pyproject.toml, o padrão moderno do Python: as dependências da aplicação, um grupo dev só com as ferramentas de desenvolvimento, e a configuração do lint (ruff), dos tipos (mypy em modo estrito) e da cobertura. O Dockerfile também lê as dependências daqui."*

# Clean Architecture: quatro camadas




**as dependências apontam para dentro**

### Regra de ouro

- domain/ não conhece banco, FastAPI nem nada externo.
- application/ só conhece o domain/.
- Um teste falha se alguém quebrar isso.

**O QUE FALAR**

*"Dividimos o código em quatro camadas. No centro fica o domínio, com as regras do negócio, que não depende de nenhum framework. Em volta, os casos de uso; depois os adaptadores, que falam HTTP e SQL; e por fora a infraestrutura. Uma camada só pode depender das de dentro — nunca o contrário."*

# Domínio rico: a regra mora na entidade

```python
class Orcamento:   # domain/entities/orcamento.py

    def mover_estagio(self, novo_estagio, hoje):
        if not self._estagio.pode_mover_para(novo_estagio):
            raise TransicaoInvalida(...)        # RN01
        if novo_estagio.exige_valor() and self._valor is None:
            raise TransicaoInvalida(...)        # RN02
        self._estagio = novo_estagio
        if novo_estagio.eh_terminal():
            self._data_fechamento = hoje        # RN03
```




qualquer estágio aberto → PERDIDO · pular etapa = erro 422

### Também são “ricos”

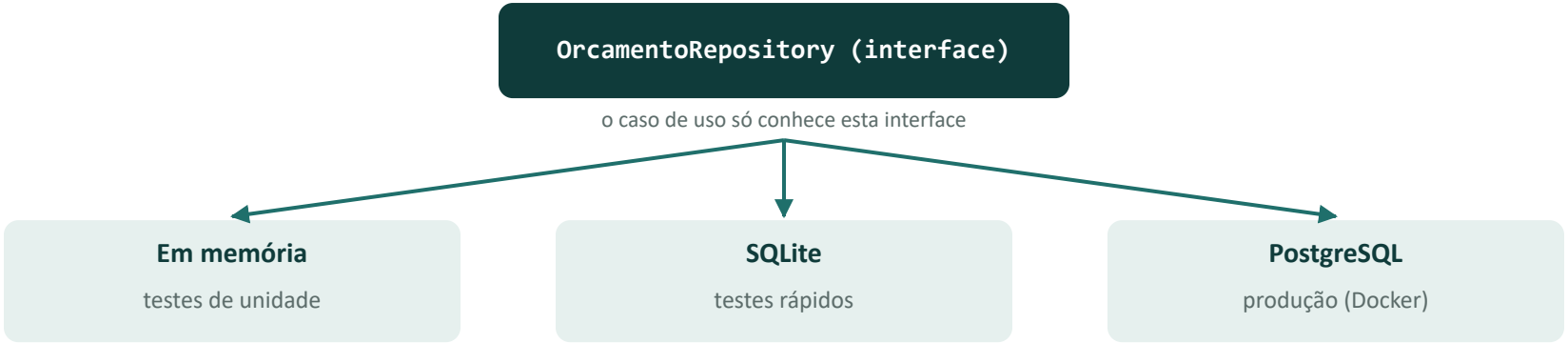
- Value Objects que se validam sozinhos: Telefone, Email, ValorOrcado.
- Interação é imutável depois de criada.
- Administrador não pode desativar a si mesmo.
- Tudo testado sem banco e sem API.

**MOSTRE: domain/entities/orcamento.py**

**O QUE FALAR**

*"Domínio rico quer dizer que a regra de negócio fica dentro da entidade, e não espalhada pelas rotas. O orçamento só anda no funil na ordem certa, só vai para 'enviado' com valor definido e marca a data de fechamento sozinho. Não importa por onde o pedido chegue — API, script ou teste — a regra vale."*

# Baixo acoplamento: trocar sem quebrar




**Prova: `tests/unit/test_arquitetura.py`**

Lê cada arquivo de domain/ e application/ e falha se encontrar um import de fastapi, sqlalchemy, pydantic, jwt... ou de uma camada mais externa. A arquitetura é verificada a cada execução dos testes.

**O QUE FALAR**

*"Os casos de uso dependem de interfaces, não do banco. Por isso a mesma suíte de testes roda com repositório em memória, SQLite e PostgreSQL sem mudar nada. E para garantir que ninguém quebre a arquitetura sem perceber, escrevemos um teste que verifica os imports de cada camada."*

# O mapeamento relacional: 4 tabelas

**usuario**

```
id · nome · email (único)
senha_hash · perfil · ativo
```

### O banco também protege os dados

- CHECK em estagio, tipo e perfil: valores fora do enum são recusados.
- Não dá para apagar cliente com orçamento (RESTRICT).
- 3 índices para o painel Kanban carregar rápido.

**cliente**

```
id · nome · telefone · email
empresa · criado_em
```

1 : N
RESTRICT

**orcamento**

```
id · cliente_id · estagio
valor · data_abertura
data_fechamento
```

1 : N
CASCADE

**interacao**

```
id · orcamento_id · tipo
conteudo · data_hora
```

**MOSTRE: pgAdmin → crm → Schemas → public → Tables**

**O QUE FALAR**

*"São quatro tabelas, mapeadas com SQLAlchemy em infra/db/models.py: usuário, cliente, orçamento e interação. Um cliente tem vários orçamentos, e cada orçamento tem várias interações. Além das regras no código, o próprio banco barra dado inválido, com CHECKs e chaves estrangeiras — e há testes provando isso."*

# Migrations versionadas com Alembic

**0001** — `esquema_inicial`

cria as 4 tabelas, CHECKs, chaves e índices

**0002** — `fechar_tabelas`

liga RLS e fecha o acesso público às tabelas

```
# a API aplica as pendentes sozinha, ao subir:
python -m alembic upgrade head

# em qual versão está este banco?
select * from alembic_version;   -- 0002
```

### Rastreabilidade

- Cada mudança no esquema é um arquivo novo, no Git, com upgrade e downgrade.
- Todo banco (dev, testes, produção) sabe em que versão está pela tabela alembic_version.
- Um teste confere que a migration gera exatamente o que os models descrevem.

**O QUE FALAR**

*"O banco não é criado à mão: cada mudança é uma migration do Alembic, versionada no Git. Quando o container da API sobe, ele roda 'alembic upgrade head' e aplica só o que falta. Assim desenvolvimento e produção ficam sempre na mesma versão, e dá para saber exatamente o que mudou e quando."*

# 14 rotas RESTful em /api/v1

| | | | | | |
|---|---|---|---|---|---|
| POST | /auth/login | entrar | PATCH | /orcamentos/{id}/estagio | mover no funil |
| GET | /auth/eu | quem sou eu | PATCH | /orcamentos/{id}/valor | definir valor |
| POST | /clientes | novo contato + orçamento | POST | /orcamentos/{id}/interacoes | registrar contato |
| GET | /clientes/{id} | dados | GET | /metricas/funil | taxa de fechamento |
| GET | /clientes/{id}/historico | linha do tempo | GET | /usuarios | listar (admin) |
| GET | /orcamentos | painel Kanban | POST | /usuarios | criar (admin) |
| GET | /orcamentos/transicoes | regras p/ front | PATCH | /usuarios/{id} | editar (admin) |

**O QUE FALAR**

*"A API segue REST: o recurso fica no caminho e o verbo diz a ação — GET consulta, POST cria, PATCH altera parte. Os códigos de resposta também seguem o padrão: 201 ao criar, 401 sem login, 403 sem permissão, 404 quando não existe, 409 conflito e 422 quando a regra de negócio recusa."*

# Swagger: documentação que executa

### Gerado automaticamente

- O FastAPI gera o OpenAPI a partir do código: não existe documento para ficar desatualizado.
- Cada entrada tem exemplo preenchido — há teste garantindo que o exemplo é aceito.
- Botão Authorize para testar as rotas protegidas.

### Validação de payload: o erro sempre no mesmo formato

```
PATCH /orcamentos/{id}/estagio
{ "estagio": "FECHADO" }      # pulando etapas

HTTP 422
{
  "erro": "TransicaoInvalida",
  "mensagem": "Não é possível mover de
              NOVO_CONTATO para FECHADO",
  "campo": "estagio"
}
```

**MOSTRE: localhost:8000/docs**

**O QUE FALAR**

*"O Swagger é gerado pelo próprio FastAPI e permite executar cada rota pelo navegador. Os dados de entrada e saída são validados por schemas Pydantic, e todo erro volta no mesmo formato, com o campo que deu problema — isso facilita muito a integração com o frontend, que pode mostrar a mensagem direto no formulário."*

# O fluxo de login com JWT

**1 — Envia**
POST /auth/login
e-mail + senha

→

**2 — Confere**
compara a senha com o hash do banco

→

**3 — Devolve token**
JWT assinado, válido por 8 horas

→

**4 — Usa o token**
Authorization: Bearer ... em cada pedido

→

**5 — Valida**
assinatura, validade e usuário ativo

| | |
|---|---|
| sem token / token inválido | 401 |
| usuário comum em /usuarios | 403 |
| token válido | 200 |

**MOSTRE: Swagger: GET /orcamentos sem login (401) → login → Authorize → de novo (200)**

**O QUE FALAR**

*"Para entrar, o usuário manda e-mail e senha uma única vez e recebe um token JWT, assinado com uma chave secreta do servidor e válido por 8 horas. Depois, só o token circula. Sem token a API responde 401; com token mas sem permissão, 403. E a cada pedido o usuário é recarregado: se for desativado, perde o acesso na hora."*

# Senhas e proteções




O banco guarda só o hash: dele não se volta à senha. Veja na tabela usuario no pgAdmin.

| | |
|---|---|
| **Sal aleatório** | senhas iguais geram hashes diferentes |
| **Lento de propósito** | adivinhar milhões de senhas fica caro |
| **Mesma resposta** | senha errada ou e-mail inexistente: mesmo 401 e mesmo tempo |
| **Segredo obrigatório** | em produção a API não sobe com o JWT_SECRET padrão |
| **Sem cadastro aberto** | só o admin cria usuários; o 1º admin é criado por script |
| **Regras do admin** | não pode se desativar nem rebaixar o próprio perfil |

**O QUE FALAR**

*"A senha nunca é guardada: o banco tem só um hash scrypt com sal, que é lento de propósito para dificultar ataques de força bruta. O login responde igual para senha errada e e-mail inexistente, para não revelar quem tem conta. E não existe cadastro público: só o administrador cria usuários."*

# A estratégia de testes

**1** — **e2e**

o funil inteiro, do cadastro ao fechamento

**216** — **Integração**

rotas + banco: SQLite e PostgreSQL

**207** — **Unidade**

domínio e casos de uso, sem banco

**MOSTRE: pasta backend/tests no VS Code**

### Tudo em tests/

- unit/: regras do domínio e casos de uso com repositório em memória — milissegundos.
- integration/: rotas HTTP, transações, integridade do banco, migrations, carga de 500 orçamentos.
- e2e/: um usuário percorrendo o funil inteiro.
- Escritos com TDD: o teste antes do código.

**O QUE FALAR**

*"São 424 testes em três níveis. Os de unidade testam as regras isoladas, sem banco, e rodam em milissegundos. Os de integração sobem a API e um banco real e testam rota, banco e serviço juntos. E um teste ponta a ponta percorre o funil inteiro. Os testes foram escritos antes do código, seguindo TDD."*

# Cobertura: meta de 80%, resultado de 100%

meta do projeto

**80%**

`fail_under = 80`

cobertura atual

**100%**

linhas e desvios

tempo no Docker

**~15 s**

423 passam, 1 pulado

```
docker compose run --rm testes
# Required test coverage of 80.0% reached. Total coverage: 100.00%
# 423 passed, 1 skipped
```

Abra backend/htmlcov/index.html: verde = testado, vermelho = não testado.

**O QUE FALAR**

*"A meta é 80% e o comando falha se cair abaixo disso. Hoje estamos em 100% — medindo não só as linhas, mas os desvios: os dois caminhos de cada if. O relatório HTML mostra o código colorido. O teste pulado confere uma proteção que só existe no PostgreSQL, então a versão dele para SQLite é pulada."*

# docker-compose orquestra, Dockerfile empacota

| | |
|---|---|
| `db` | **PostgreSQL 17**<br>volume pgdata guarda os dados |
| `api` | **backend FastAPI**<br>porta 8000 · roda as migrations |
| `pgadmin` | **banco no navegador**<br>porta 5050 |
| `testes` | **suíte + cobertura**<br>só com docker compose run |

```
FROM python:3.14-slim AS base
COPY pyproject.toml .
RUN pip install <dependências do pyproject>
COPY alembic src ...

FROM base AS testes      # + grupo dev
CMD pytest --cov

FROM base AS api         # usuário sem root
CMD alembic upgrade head && uvicorn ...
```

**O QUE FALAR**

*"O docker-compose sobe tudo com um comando: o banco PostgreSQL, a API, o pgAdmin e, quando pedido, os testes. A API só sobe depois que o banco está saudável. O Dockerfile tem duas versões: uma para rodar a API, com um usuário sem privilégio, e uma para rodar os testes. Qualquer máquina com Docker roda o projeto igual."*

# Destaques: por que é uma boa arquitetura

### 1 Escalável

A API não guarda sessão (JWT). Dá para rodar várias cópias atrás de um balanceador; o painel com 500 orçamentos responde bem abaixo do limite de 2 s.

### 2 Segura

Hash scrypt, JWT com validade, 401/403, CORS restrito, segredos no .env fora do Git, banco acessível só nesta máquina.

### 3 Boas práticas

Clean Architecture, DDD, TDD, tipagem estrita (mypy), lint (ruff) e regras de negócio fora do framework.

### 4 Testada e documentada

100% de cobertura, Swagger automático, README com como rodar e um plano do backend em docs/.

**O QUE FALAR**

*"Para fechar: a API é stateless, então está pronta para escalar horizontalmente. É segura nas senhas e no acesso. Segue boas práticas que são verificadas automaticamente — tipos, lint, arquitetura e cobertura. E está documentada para quem continuar o projeto, inclusive o frontend."*

DEMONSTRAÇÃO AO VIVO

# Roteiro da demo (cerca de 5 minutos)

| | | | |
|---|---|---|---|
| 1 | Swagger | **GET /orcamentos sem login** | 401 — a rota está protegida |
| 2 | Swagger | **POST /auth/login → Authorize** | entrou: agora tem token |
| 3 | Swagger | **POST /clientes (exemplo pronto)** | 201 — cliente + orçamento em NOVO_CONTATO |
| 4 | pgAdmin | **F5 na consulta** | o cliente apareceu no banco |
| 5 | Swagger | **PATCH .../estagio com FECHADO** | 422 — o domínio barrou o pulo |
| 6 | Swagger | **EM_NEGOCIACAO → valor → ENVIADO → FECHADO** | 200 em cada passo |
| 7 | pgAdmin | **F5 na consulta** | FECHADO, com valor e data de fechamento |
| 8 | Swagger | **GET /metricas/funil** | taxa de fechamento calculada |

**O QUE FALAR**

*Se sobrar tempo: rode "docker compose run --rm testes" ao vivo — em uns 15 segundos aparecem os 423 testes e os 100% de cobertura. O erro 422 é o momento mais forte da demo: mostra a regra de negócio protegendo o banco.*

ESTEJA PRONTO

# Perguntas que o professor pode fazer

| | |
|---|---|
| **Por que regra no domínio e não na rota?** | Para valer em qualquer caminho e ser testada sem banco nem framework. |
| **Por que JWT e não sessão?** | O servidor não guarda estado: escala melhor e funciona bem com o frontend separado. |
| **Por que scrypt?** | É próprio para senhas (lento e com sal) e já vem no Python. |
| **E se o token vazar?** | Expira em 8 h; desativar o usuário corta o acesso na hora. |
| **Por que SQLite nos testes?** | Roda em qualquer máquina; as mesmas provas também rodam no PostgreSQL. |
| **Para que a migration 0002?** | Fecha a API pública que o Supabase cria; no Docker só liga a proteção (RLS). |
| **100% de cobertura garante zero bug?** | Não: garante que tudo foi executado. Por isso também testamos regras e casos de erro. |

COLA

# Comandos e endereços

```
# na raiz Projeto_CRM, com o Docker Desktop aberto
docker compose up -d                 # liga banco, API e pgAdmin
docker compose ps                    # confere se está tudo running
docker compose run --rm testes       # testes + cobertura (~15 s)
docker compose logs api              # se algo der errado
docker compose down                  # desliga (os dados ficam)

# lint e tipos, pelo Docker
docker compose run --rm testes python -m ruff check src tests
docker compose run --rm testes python -m mypy
```

**Swagger**

`localhost:8000/docs`

**pgAdmin**

`localhost:5050`

**Cobertura**

`backend/htmlcov/index.html`

**Plano**

`docs/plano-backend.md`

**Frase de encerramento**

*"O backend implementa o funil de atendimento com as regras de negócio no domínio, protegido por login e JWT, com banco versionado pelo Alembic, documentado no Swagger, 100% coberto por testes e pronto para rodar em qualquer máquina com um docker compose up."*